

16-891 Multi Robot Planning and Coordination

Instructor : Jiaoyang Li TA: Vaibhav Sanjay

Fall 2026 Coding Assignment 1: Multi-Agent Path Finding (MAPF)

Assignment Deadline: 11:59 PM, September 16th 2026

In this assignment, you will learn about Multi-Agent Path Finding (MAPF) and implement a single-agent solver, namely space-time A*, and parts of four MAPF solvers, namely Joint-State A*, Prioritized Planning, Conflict-Based Search (CBS), and Priority-Based Search (PBS). Skeleton code is given to make the assignment easier and more structured.

Honor Code

As a student, you may discuss with your peers, but at no point should you seek assistance on your code from them, from any generative AI tools, or from the internet. Your code submission should be yours and yours only. This assignment has a lot of learning and intuition waiting to be acquired if only you followed this Honor code and remained loyal to your learning!

Grading Rubrics

This assignment has 20 points in total: 12 for the coding part and 8 for the write-up. For the write-up part, the point distribution is included in the respective task sections. For the coding part, the point distribution and the number of test cases on Gradescope are given in the following table.

Algorithm	Points	Test Cases
Joint-State A*	2	5
Prioritized Planning	2	5
Conflict-Based Search	4	20
Priority-Based Search	4	20

To get all points for each path-finding solver, your implementations must pass all corresponding test cases on Gradescope. For Joint-State A* and Conflict-Based Search, your solvers are expected to find optimal paths with the minimum sum of costs. Some test cases (`instances/test_*.txt`) are released with the code template for validating your implementation before submission.

Submission Details

Code submission: Compress the following files into one .zip file and upload to Gradescope.

- `single_agent_planner.py`
- `joint_state.py`
- `prioritized.py`

- `cbs.py`
- `pbs.py`

Important: Do not place these files inside a subfolder; the files should be at the top level of the zip archive.

Write-up submission: Fill in `write-up-template.tex` and upload the PDF version to Gradescope.

0 Task 0: Preparing for the Project

0.1 Installing Python 3

This project requires a Python 3 installation with the `numpy` and `matplotlib` packages. On Ubuntu Linux, download Python by using:

```
sudo apt install python3 python3-numpy python3-matplotlib
```

On Mac OS X, download Anaconda 2019.03 with Python 3 from <https://www.anaconda.com/distribution/#download-section> and follow the installer. You can verify your installation by using:

```
python3 --version
```

On Windows, download Anaconda 2019.03 with Python 3 from <https://www.anaconda.com/distribution/#download-section>.

On Ubuntu Linux and Mac OS X, use `python3` to run python. On Windows, use `python` instead.

0.2 Installing the MAPF Software

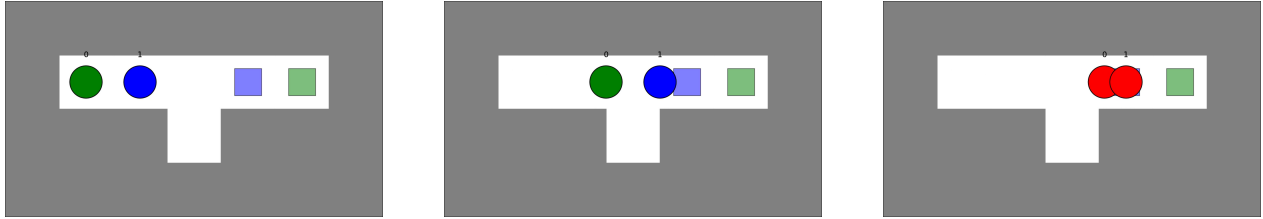
By cloning the [GitHub repository](#), you should have all the necessary files to start the assignment. The main driver code sits in `run_experiments.py`. Some helper functions are provided along the way. **In addition, your next assignment will be built based on your solution to assignment 1. Therefore, it is recommended to not change anything unless the section has a TODO.**

0.3 Understanding Independent Planning

Execute the independent MAPF solver by using:

```
python run_experiments.py --instance instances/exp0.txt --solver Independent
```

If you are successful, you should see an animation:



The independent MAPF solver plans for all agents independently. Their paths do not collide with the environment but are allowed to collide with the paths of the other agents. Thus, there is a collision when the blue agent 1 stays at its goal cell while the green agent 0 moves on top of it. In your animation, both agents turn red when this happens, and a warning is printed on the terminal notifying you about the details of the collision.

Try to understand the independent MAPF solver in `independent.py`. The first part defines the class `IndependentSolver` and its constructor:

```
class IndependentSolver(object):
    def __init__(self, my_map, starts, goals):
        # some parts are omitted here for brevity
        # compute heuristic values for the A* search
        self.heuristics = []
        for goal in self.goals:
            self.heuristics.append(compute_heuristics(my_map, goal))
```

The function `compute_heuristics` receives as input the representation of the environment and the goal cell of the agent and computes a look-up table with heuristic values (or, synonymously, h-values) for the A* search that finds a path for the agent, by executing a Dijkstra search starting at the goal cell.

The second part performs one A* search per agent:

```
def find_solution(self):
    for i in range(self.num_of_agents): # Find path for each agent
        path, expanded = a_star(self.my_map, self.starts[i], self.goals[i],
                                ↪ self.heuristics[i], i, [])
        self.a_star_nodes_expanded += expanded
        if path is None:
            raise BaseException('No solutions')
        result.append(path)
    return result
```

The function `a_star` receives as input the representation of the environment, the start cell of the agent, the goal cell of the agent, the heuristic values computed in the constructor, the unique agent id of the agent, and a list of constraints and performs an A* search to find a path for the agent. The independent MAPF solver does not use constraints.

1 Task 1: Implementing Joint-State A* and Space-Time A*

Section 1.1 covers Joint-State A* whereas Sections 1.2 to 1.4 are for Space-Time A*. Have fun!

1.1 Implementing Joint-State A*

A simple method for planning for (very) small teams of agents in a centralized setting is joint state space planning where we plan for M agents in a state space that represents joint configurations of agents (also called composite state space). The approach is simple: Construct and search a graph where each state encodes the positions of all the agents and each action encodes all possible movements. You will now complete the `joint_state_a_star` function inside `single_agent_planner.py` and finish `joint_state.py`. Some prompts are given in the code to help your implementation.

You can test your Joint-State A* implementation by using:

```
python run_experiments.py --instance instances/exp0.txt --solver JointState
```

Expected Results: It is expected to find a solution for `exp0.txt` with a sum of costs equal to 8.

(Optional) You can also try running Joint-State A* with more complex test cases such as `instances/test_21.txt`. If the solver takes too long to find solutions, you can try to reduce the number of agents.

1.2 Searching in the Space-Time Domain

You may observe that Joint-State A* slows down drastically as the number of agents increases. It's time to upgrade our algorithms! To prepare for that, you now change the single-agent solver to perform a space-time A* search, which searches in cell-time space and returns a shortest path that satisfies a given set of constraints. Such constraints are essential for MAPF solvers such as prioritized planning and CBS.

The existing A* search in the function `a_star` in `single_agent_planner.py` only searches over cells. Since we want to support temporal constraints, we also need to search over time steps. Use the following steps to change the search dimension:

1. Your variables `root` and `child` are dictionaries with various key/value pairs such as the g-value, h-value, and cell. Add a new key/value pair for the time step. The time step of the root node is zero. The time step of each node is one larger than that of its parent node.
2. The variable `closed_list` contains the expanded nodes. Currently, this is a dictionary indexed by cells. Use tuples of (cell, time step) instead.
3. When generating child nodes, do not forget to add a child node where the agent waits in its current cell instead of moving to a neighboring cell.

You can test your code by using:

```
python run_experiments.py --instance instances/exp1.txt --solver Independent
```

Expected Results: Since inter-robot collision avoidance has not been considered yet, you are expected to see a solution that contains collisions and has a sum of costs of 6.

1.3 Handling Vertex Constraints

We first consider vertex constraints, that prohibit a given agent from being in a given cell at a given time step. Each constraint is a Python dictionary. The following code creates a vertex constraint that prohibits agent 1 from occupying cell (1,2) at time step 3:

```
{'agent': 1,  
 'loc': [(1,2)],  
 'timestep': 3}
```

In order to add support for constraints, change the code to check whether the new node satisfies the constraints passed to the `a_star` function and prune it if it does not.

An efficient way to check for constraint violations is to create, in a pre-processing step, a constraint table, which indexes the constraints by their time steps. At runtime, a lookup in the table is used to verify whether a constraint is violated. Example function headers for the functions `build_constraint_table` and `is_constrained` are already provided. You can call `build_constraint_table` before generating the root node in the `a_star` function.

You can test your code by adding a constraint in `prioritized.py` that prohibits agent 0 from being at its goal cell (1,5) at time step 4 and then using:

```
python run_experiments.py --instance instances/exp1.txt --solver Prioritized
```

Expected Results: Agent 0 should wait for one time step (but when and where it waits depends on the tie-breaking).

1.4 Adding Edge Constraints

We now consider edge constraints, that prohibit a given agent from moving from a given cell to another given cell at a given time step.

The following code creates a edge constraint that prohibits agent 2 from moving from cell (1,1) to cell (1,2) from time step 4 to time step 5:

```
{'agent': 2,  
 'loc': [(1,1), (1,2)],  
 'timestep': 5}
```

Implement constraint handling for edge constraints in the function `is_constrained`.

You can test your code by adding a constraint in `prioritized.py` that prohibits agent 1 from moving from its start cell (1,2) to the neighboring cell (1,3) from time step 0 to time step 1.

2 Task 2: Implementing Prioritized Planning

Prioritized planning finds paths for all agents, one after the other, that do not collide with the environment or the already planned paths of the other agents. To ensure that the path of an agent does not collide with the already planned paths of the other agents, the function `a_star` receives as input a list of constraints compiled from their paths. Prioritized planning is faster than CBS but it is incomplete and suboptimal.

2.1 Adding Vertex and Edge Constraints

Add code to `prioritized.py` that adds all necessary vertex and edge constraints. Transform the already planned paths of higher-priority agents into constraints.

You can test your code by using:

```
python run_experiments.py --instance instances/exp2_1.txt --solver Prioritized
```

Expected Results: It is expected to find a solution for `exp2_1.txt`.

2.2 Adding Additional Constraints

Now try the following test cases:

```
python run_experiments.py --instance instances/exp2_2.txt --solver Prioritized
python run_experiments.py --instance instances/exp2_3.txt --solver Prioritized
python run_experiments.py --instance instances/exp2_4.txt --solver Prioritized
```

Expected Results: You will notice that just vertex and edge constraints as defined above are not sufficient to handle a few edge cases. We will tackle them now.

2.2.1 Case 1: Higher-Priority Agents Passing One’s Goal Position in the “Future”

At this point, the A^* search terminates when a node containing the goal position is expanded for the first time. However, there are cases where after an agent reaches its goal position, it would still need to move away from the goal position temporarily to give ways for other higher-priority agents to pass in the “future”. For example, let’s assume agent 0 reaches its goal position at timestep 2. And at timestep 10, agent 1 with a higher priority needs to pass through agent 0’s goal position en route to its own destination. Therefore, agent 0 cannot be at the goal position at timestep 10. To handle such scenarios, the A^* search for agent 0 cannot terminate right away at timestep 2, and a vertex constraint needs to be added for agent 0 at timestep 10. Modify your code to handle such scenarios, where the reaching-goal condition check considers future constraints imposed by other agents with a higher priority. Verify that your solution works by testing on `exp2_2.txt`.

2.2.2 Case 2: Higher-Priority Agents Staying at Their Goal Positions

Your code does not prevent all collisions yet since agents can still move on top of other agents that have already reached their goal locations. You can verify this issue by using the MAPF instance `exp2_3.txt` and assuming that agent 0 has the highest priority. You can address this issue by adding code that adds additional constraints that apply not only to the time step when agents reach their goal locations but also to all future time steps. One way to achieve that is by limiting the time horizon of the search. The shortest path of an agent cannot be infinitely long. So you can calculate an upper bound on the path length for an agent based on the path lengths of all agents with higher priorities and the size of the environment. Then you can simply add vertex constraints at a higher-priority agent’s goal position from the moment it reaches its goal position throughout the entire time horizon. Another way to address this is by switching back to the vanilla A^* (without the time dimension and the “wait” action) after all other higher-priority agents reach their goals. Test your implementation on `exp2_3.txt`.

2.2.3 Case 3: Addressing Failures

The A* search might not be able to find a collision-free solution in some cases. Your implementation needs to handle such failure cases properly by terminating the A* search at the right time. Make sure your solver terminates properly and reports “no solutions”. You may test your implementation on `exp2_4.txt`.

(3 points) Answer the following question in your report: Document the changes you made in your code to handle the above three cases. Also, state and justify any upper bound or termination criteria you used in your code (i.e., why they can address the issues and why they preserve the optimality and completeness of the A* search).

3 Task 3: Implementing Conflict-Based Search (CBS)

The independent MAPF solver finds paths for all agents, simultaneously or one after the other, that do not collide with the environment but are allowed to collide with the paths of the other agents. Conflict-Based Search (CBS) maintains a constraint tree to deal with these collisions between agents.

3.1 Detecting Collisions

Write code that detects collisions (or, synonymously, conflicts) among agents, namely vertex collisions where two agents are in the same cell at the same time step and edge collisions where two agents move to the cell of the other agent at the same time step.

Add code to `cbs.py` that implements the two functions `detect_first_collision_for_path_pair` and `detect_collisions_among_all_paths`. You should use `get_location(path,t)` to obtain the cell of an agent at time step t . You can test your code by using:

```
python run_experiments.py --instance instances/exp3_1.txt --solver CBS
```

Expected Results: You receive output similar to

```
[{'a1': 0, 'a2': 1, 'loc': [(1, 4)], 'timestep': 3}]
```

3.2 Converting Collisions to Constraints

The high level of CBS searches the constraint tree. Once it has chosen a node of the constraint tree for expansion and picked a collision of the paths of two agents in that node, it transforms this collision into two new constraints, one for each new child node of the chosen node. The first constraint prohibits the first agent from executing the colliding action, and the second constraint prohibits the second agent from executing the colliding action. Add code to `cbs.py` that implements the function `standard_splitting` and test your code as above.

Expected Results: For the vertex collision between agents 1 and 2 in cell (1,4) at time step 3, the set of new vertex constraints is:

```
[{'agent': 0, 'loc': [(1, 4)], 'timestep': 3},  
 {'agent': 1, 'loc': [(1, 4)], 'timestep': 3}]
```

Algorithm 1: High-level search of CBS.

Input: Representation of the environment, start cells, and goal cells
Result: optimal collision-free solution

```
1  $R.constraints \leftarrow \emptyset$ 
2  $R.plan \leftarrow$  find independent paths for all agents using  $a\_star()$ 
3  $R.collisions \leftarrow detect\_collisions(R.paths)$ 
4  $R.cost \leftarrow get\_sum\_of\_cost(R.paths)$ 
5 insert  $R$  into OPEN
6 while  $OPEN \neq \emptyset$  do
7    $P \leftarrow$  node from OPEN with the smallest cost
8    $OPEN \leftarrow OPEN \setminus \{P\}$ 
9   if  $P.collisions = \emptyset$  then
10    return  $P.plan$  //  $P$  is a goal node
11    $collision \leftarrow$  one collision in  $P.collisions$ 
12    $constraints \leftarrow resolve\_collision(collision)$ 
13   for  $constraint$  in  $constraints$  do
14      $Q \leftarrow$  create copy of  $P$ 
15      $Q.constraints \leftarrow P.constraints \cup \{constraint\}$ 
16      $a_i \leftarrow$  the agent in  $constraint$ 
17      $path \leftarrow a\_star(a_i, Q.constraints)$ 
18     if  $path \neq \emptyset$  then
19       Replace the path of agent  $a_i$  in  $Q.plan$  by  $path$ 
20        $Q.collisions \leftarrow detect\_collisions(Q.plan)$ 
21        $Q.cost \leftarrow get\_sum\_of\_cost(Q.plan)$ 
22       Insert  $Q$  into OPEN
23 return 'No solutions'
```

3.3 Implementing the High-Level Search

Algorithm 1 shows the pseudo-code of the high-level search of CBS. Add code to `cbs.py` that finalizes the high-level search of CBS in the function `find_solution`, where we have already provided the implementation of lines 1-5. To manage the OPEN list, you can use the helper functions `push_node` and `pop_node`. Add print statements that list the expanded nodes (for debugging), and test your code as before.

Hint: In Python, to copy the parent node, use `copy.deepcopy` to ensure the code functions as intended.

3.4 Testing Your Implementation

You can test your implementation by running it on our test instances:

```
python run_experiments.py --instance "instances/test_*" --solver CBS --batch
```

(This may take a while depending on your computer.) The batch command creates an output file `results.csv`, which you can compare to the one provided in `instances/min-sum-of-costs.csv`.

Expected Results: With a time limit of 300 seconds, you are expected to find solutions for all test cases except `test_20.txt`, which may result in a timeout. Depending on your machine, `test_14.txt` may also time out. For all test cases where a solution is found, the reported sum of costs should match the values in `instances/min-sum-of-costs.csv`.

4 Task 4: Implementing Priority-Based Search (PBS)

Priority-Based Search (PBS) generalizes the notion of prioritized planning with a fixed total priority ordering on the agents to planning with all possible total priority orderings. It performs a depth-first search on the high level to dynamically construct a priority ordering and thus builds a priority tree (PT). PBS computes near-optimal solutions and is much more efficient than CBS.

4.1 Generating Priority Pairs

When a collision occurs between two agents, PBS explores both scenarios where one is prioritized over the other and vice versa. In particular, when PBS expands a PT node N to resolve a collision, PBS generates two child PT nodes N_1 and N_2 that correspond to the ordered priority pairs $j \prec i$ and $i \prec j$. Add code to `pbs.py` that implements the function `generate_priority_pairs`.

4.2 Implementing the High-level Search

Algorithm 2 shows the pseudo-code of the high-level search of PBS. Add code to `pbs.py` that finalizes the high-level search of PBS in the function `find_solution` and the function `update_plan`. You may want to make use of the code from Task 3.3 and the following utility functions:

- `get_lower_priority_agents/get_higher_priority_agents` give you the agents with lower/higher priority compared to a given agent in a topological ordering.
- `collide_with_higher_priority_agents` determines whether a given agent's path collides with the paths of other agents that have a higher priority in a topological ordering.

Algorithm 2: High-level search of PBS.

Input: Representation of the environment, start cells, and goal cells, $\prec_0$ ($= \emptyset$ by default)

```
1  $R.priority\_pairs \leftarrow \prec_0$ 
2  $R.plan \leftarrow$  find independent paths for all agents using  $a\_star()$ 
3  $R.collisions \leftarrow detect\_collisions(R.plan)$ 
4  $R.cost \leftarrow get\_sum\_of\_cost(R.plan)$ 
5  $STACK \leftarrow \{R\}$ 
6 while  $STACK \neq \emptyset$  do
7    $P \leftarrow$  top node in  $STACK$ 
8    $STACK \leftarrow STACK \setminus \{P\}$ 
9   if  $P.collisions = \emptyset$  then
10     $\mid$  return  $P.plan$ 
11    $collision \leftarrow$  a vertex or edge collision  $\langle a_i, a_j, \dots \rangle$  in  $P.collisions$ 
12   for  $a_i$  involved in  $collision$  do
13      $Q \leftarrow$  create copy of  $P$ 
14      $Q.priority\_pairs \leftarrow P.priority\_pairs.append((j, i))$ 
15      $success \leftarrow update\_plan(Q, a_i)$ 
16     if  $success$  then
17        $Q.cost \leftarrow get\_sum\_of\_cost(Q.paths)$ 
18        $Q.collisions \leftarrow detect\_collisions(Q.paths)$ 
19   Insert new nodes  $Q$  into  $STACK$  in non-increasing order of  $Q.cost$ .
20 return 'No solutions'
21 Function  $update\_plan(N, a_i)$ :
22    $LIST \leftarrow$  topological sorting on partially ordered set  $(\{i\} \cup \{j \mid i \prec_N j\})$  //  $i$  and all  $j$ s
   that are having lower priorities than  $i$  specified by  $N.priority\_pairs$ 
23   for  $j \in LIST$  do
24     if  $j = i$  or  $a_j$  collides with  $a_k$  in  $N.plan$ , where  $k \prec_N j$  then
25       Update  $a_j$ 's path in  $N.plan$  by invoking a low-level search for  $a_j$ , where the paths of
        $a_k$  ( $k \prec_N j$ ) are converted into constraints for  $a_j$ 
26       if no path returned by the low-level search then
27          $\mid$  return  $false$ 
28   return  $true$ 
```

4.3 Testing Your Implementation

You can test your implementation by running it on our test instances:

```
python run_experiments.py --instance "instances/test_*" --solver PBS --batch
```

(This may take a while depending on your computer.) The batch command creates an output file `results.csv`, which you can compare to the one provided in `instances/min-sum-of-costs.csv`.

Expected Results: With a time limit of 300 seconds, you are expected to find solutions for all test cases. The reported sum of costs should be greater than or equal to the values in `instances/min-sum-of-costs.csv`.

5 Task 5: Challenge Your MAPF Solvers

Time for challenges! In this problem, you will run your Prioritized Planning (Task 2), CBS (Task 3), and PBS (Task 4) solvers on two maps and compare their performance.

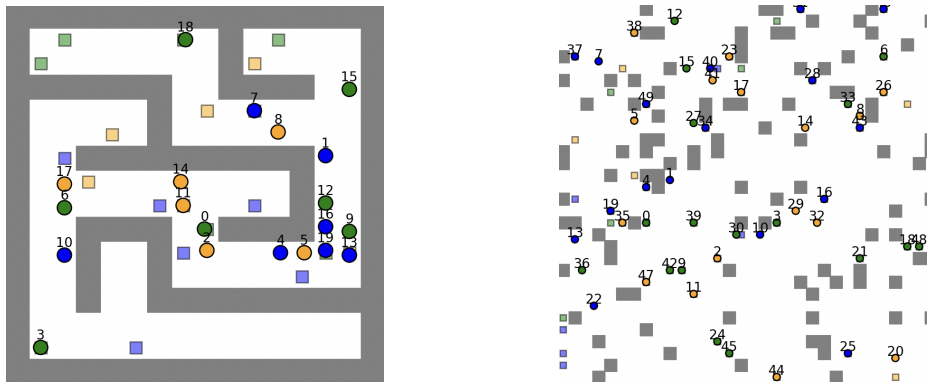


Figure 1: Test your implementation on these complex maps and see how well they run

5.1 Maze Map

Evaluate each of the three solvers on `maze_map.txt` with increasing agent counts in steps of 2 (i.e., 2, 4, 6, ...). Start with 2 agents by setting the agent count in `maze_map.txt` to 2 (you don't need to delete the start/goal positions). Test all three solvers, then incrementally add 2 to the agent count.

The cases can be quite hard for some solvers to find a solution in a reasonable amount of time, hence **limit your algorithms to 200,000 A* node expansions**. If a solver fails to find a solution for an instance, you do not need to test that solver for higher agent counts.

Record the sum of costs and A* node expansions used for every successful run. The template code includes a utility function `print_results` that gives you these stats for a given solver.

(2 points) Do the following question in your report: Generate a table showing **Sum of Costs** and **A* node expansions** for each algorithm and agent count. Place a dash at any location where planning failed, or was not tested due to an earlier failure.

5.2 Random Map

Repeat the process on `random_map.txt`. However, this time **increment by 5 agents** at a time.

(2 points) Do the following question in your report: Generate the same table for `random_map.txt`.

5.3 Analysis

(1 point) Answer the following question: Analyze the performance of the 3 algorithms on these maps. Explain the differences in instances solved, A* nodes expanded, and sum of costs.

Note: The random map comes from the [Moving AI MAPF Benchmark](#) [1]. This benchmark might be useful for your class project.

References

- [1] Roni Stern, Nathan R. Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne T. Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, Eli Boyarski, and Roman Bartak. Multi-agent pathfinding: Definitions, variants, and benchmarks. *Symposium on Combinatorial Search (SoCS)*, pages 151–158, 2019.